

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 2 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	R KALAI AMUDHAN	Reg. No.:	192511292
Date:		Duration:	60 minutes

Code of Conduct

I R Kalai Amudhan (192511292) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: R KALAI AMUDHAN

Annexure A – Code Review & Repository Evaluation

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a **Code Review and Repository Evaluation** of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions

1. Problem Overview

- Explain the assigned problem statement and summarize the implemented solution.
- Describe the project objectives and key functionalities.

2. Repository Organization

- Evaluate the repository structure, folder organization, and file naming conventions.
- Justify how the repository layout supports maintainability and collaboration.

3. Code Quality Review

- Review the source code for readability, modularity, coding standards, and documentation.
- Identify strengths and areas for improvement.

4. Version Control Evaluation

- Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.
- Explain how version control supports collaborative software development.

5. Code Testing and Validation

- Demonstrate that the application functions correctly through appropriate testing.
- Present test cases, execution results, and screenshots as evidence.

6. **Repository Documentation**

- Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.
- Suggest improvements to enhance usability and maintainability.

7. **Code Review Findings and Recommendations**

- Summarize the overall code review findings.
- Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15 %)	README, installation guide, usage instructions, and documentation are complete, clear, and well organized.	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.
Review Findings, Recommendations & Presentation(10 %)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

1. PROBLEM OVERVIEW

The Intelligent AI-Based Smart Dairy Farm Management System is a FastAPI-based prototype intended to support modern dairy-farm operations through centralized health, feeding, milk-production, and analytics services. The problem statement focuses on the difficulty of continuously monitoring cattle health and farm operations using manual observation alone.

The implemented prototype provides REST endpoints for health status, cattle information, prototype disease-risk prediction, feeding recommendations, milk recording, and farm analytics. The application uses Pydantic models for structured request validation and FastAPI for API routing and automatic documentation.

Important scope note: the current disease prediction implementation is threshold-based decision logic. It demonstrates the API workflow for AI-assisted health monitoring but is not a trained machine-learning model and should not be interpreted as clinical diagnosis.

1.1 Implemented Solution

FastAPI REST backend with a clear application entry point.

Structured SensorReading and MilkRecord request models using Pydantic.

Health-check endpoint for basic application availability verification.

Cattle endpoint providing sample livestock records and milk information.

Disease-risk endpoint using temperature, heart rate, rumination, and activity thresholds.

Feeding endpoint returning a prototype feeding schedule and resource recommendation.

Milk endpoint for recording a production event.

Analytics endpoint returning farm-level summary indicators.

Dockerfile and Docker Compose configuration for reproducible deployment.

1.2 Objectives and Key Functionalities

1. Continuously monitor cattle health indicators.
2. Support early disease-risk identification.
3. Provide feeding recommendations.
4. Record milk production.
5. Provide farm performance summaries.
6. Establish a foundation for future IoT and trained AI integration.
7. Support maintainable software development through Git and Docker.

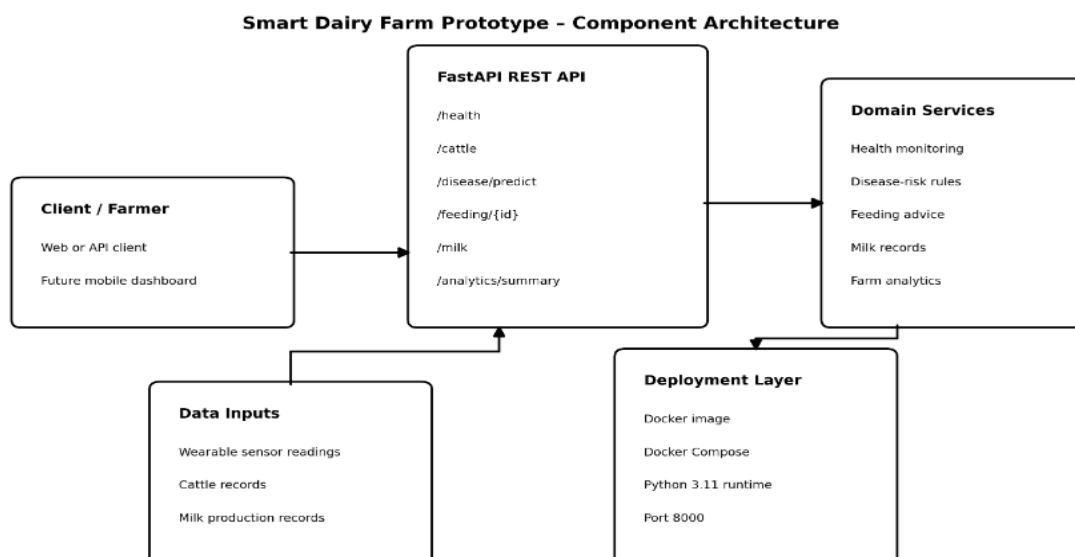


Figure 1.1 – Smart Dairy Farm Prototype Component Architecture

2. REPOSITORY ORGANIZATION

The repository was evaluated using the actual project files and a local Git history created for the prototype. The current layout is intentionally small and easy to understand, which is appropriate for a prototype. However, the repository will need additional modules as the system grows.

Current repository structure:

```
smart-dairy-project/
├── app/
│   └── main.py
├── screenshots/
├── Dockerfile
├── docker-compose.yml
├── requirements.txt
└── README.md
```

2.1 Folder and File Evaluation

Item	Current Role	Review
app/main.py	FastAPI application and endpoints	Functional, but business logic is concentrated in one file.
requirements.txt	Python dependencies	Simple and readable; versions are not pinned.
Dockerfile	Container image instructions	Clear prototype configuration.
docker-compose.yml	Application service definition	Appropriate for the current single-service prototype.
README.md	Setup and API usage information	Useful baseline; should be expanded with architecture, testing, screenshots, and deployment notes.
screenshots/	Validation evidence	Useful for assessment traceability; should normally be excluded from production source repositories if generated artifacts are not required.

2.2 Naming Convention Review

File names are descriptive and use conventional lowercase naming. Endpoint names are also understandable and resource-oriented. The main improvement would be to split the application into modules so that names communicate domain responsibility, for example routers, services, schemas, and models.

2.3 Maintainability and Collaboration

The small repository is easy for a new contributor to understand.

Docker configuration is separated from application source code.

A README provides a single starting point for installation and usage.

A modular architecture would reduce merge conflicts when several developers work on separate dairy functions.

A tests/ directory should be added so validation becomes part of the repository rather than only a report artifact.

```
Smart Dairy Repository Structure Review

$ git status --short --branch

$ git ls-files
```

Figure 2.1 – Repository Structure and Git-Tracked Files Review

3. CODE QUALITY REVIEW

The source code was reviewed for readability, modularity, coding standards, documentation, validation, maintainability, and suitability for future extension. The current code is understandable and compact, but several improvements are recommended before production deployment.

3.1 Code Readability

The application uses straightforward function names such as health, cattle, predict_disease, feeding, add_milk, and analytics_summary. The code is short enough to understand quickly, and the Pydantic models clearly identify the structure of sensor and milk requests.

3.2 Modularity

The main weakness is that the complete application is concentrated in app/main.py. Routing, domain rules, sample data, response construction, and analytics values are all located in the same module. This is acceptable for a prototype but will become difficult to maintain as more dairy features are added.

Recommended future structure:

```
app/
├── main.py
├── routers/
│   ├── health.py
│   ├── cattle.py
│   ├── disease.py
│   ├── feeding.py
│   ├── milk.py
│   └── analytics.py
├── schemas/
├── services/
├── models/
├── database/
└── tests/
```

3.3 Coding Standards

Use type annotations consistently for function parameters and return values.

Use constants or configuration objects for disease thresholds instead of hard-coded values.

Add docstrings to public endpoint functions and important domain services.

Use consistent formatting through a tool such as Black and linting through Ruff or Flake8.

Use structured logging instead of relying only on server output.

Move sample records and farm constants into data or configuration modules.

3.4 Input Validation and Error Handling

Pydantic provides basic type validation for the two request models, which is a strength. However, the models do not currently define value constraints such as non-negative temperature ranges, valid heart-rate ranges, positive milk volume, or permitted session values. Endpoint-level handling for missing cattle IDs and invalid records should also be added.

3.5 Documentation in Code

The code is readable but has limited inline documentation. Short docstrings describing endpoint purpose, expected input, and output would improve maintainability. The automatic FastAPI OpenAPI documentation is a strong supporting feature.

Source Code Review - app/main.py

```
$ source review: app/main.py
001 | from fastapi import FastAPI
002 | from pydantic import BaseModel
003 | from datetime import date
004 |
005 | app = FastAPI(title='Intelligent AI-Based Smart Dairy Farm Management System', version='1.0.0')
006 |
007 | class SensorReading(BaseModel):
008 |     cattle_id: str
009 |     temperature_c: float
010 |     heart_rate_bpm: int
011 |     rumination_minutes: int
012 |     activity_score: float
013 |
014 | class MilkRecord(BaseModel):
015 |     cattle_id: str
016 |     litres: float
017 |     session: str
018 |
019 | @app.get('/')
020 | def root():
021 |     return {'message': 'Intelligent AI-Based Smart Dairy Farm Management System'}
022 |
023 | @app.get('/health')
024 | def health():
025 |     return {'status': 'healthy', 'service': 'Smart Dairy Farm Management System', 'version': '1.0.0'}
026 |
027 | @app.get('/cattle')
028 | def cattle():
029 |     return [
030 |         {'id': 'C001', 'name': 'Lakshmi', 'status': 'Normal', 'last_milk_litres': 18.5},
031 |         {'id': 'C002', 'name': 'Nandini', 'status': 'Monitor', 'last_milk_litres': 15.2},
032 |         {'id': 'C003', 'name': 'Ganga', 'status': 'Normal', 'last_milk_litres': 20.1},
033 |     ]
034 |
035 | @app.post('/disease/predict')
036 | def predict_disease(r: SensorReading):
037 |     risk = 'Low'
038 |     prediction = 'No immediate disease risk detected'
039 |     if r.temperature_c >= 39.5 or r.heart_rate_bpm >= 100:
040 |         risk = 'High'; prediction = 'Possible fever or infection - veterinary review recommended'
041 |     elif r.rumination_minutes < 300 or r.activity_score < 0.4:
042 |         risk = 'Medium'; prediction = 'Possible digestive or activity-related issue - monitor closely'
043 |     return {'cattle_id': r.cattle_id, 'risk_level': risk, 'prediction': prediction}
044 |
045 | @app.get('/feeding/{cattle_id}')
046 | def feeding(cattle_id: str):
047 |     return {'cattle_id': cattle_id, 'recommended_feed_kg': 8.5, 'water_litres': 55, 'schedule': ['06:00', '13:00', '19:00']}
048 |
049 | @app.post('/milk')
050 | def add_milk(r: MilkRecord):
051 |     return {'message': 'Milk production recorded', 'cattle_id': r.cattle_id, 'litres': r.litres, 'session': r.session}
052 |
053 | @app.get('/analytics/summary')
054 | def analytics_summary():
055 |     return {'date': str(date.today()), 'total_cattle': 120, 'healthy_cattle': 112, 'monitoring_required': 8, 'average_milk_litres_per_cow': 17.8, 'es
```

Figure 3.1 – Source Code Review of app/main.py

4. VERSION CONTROL EVALUATION

The repository was evaluated using a local Git history created for the prototype. The history contains an initial application setup, project documentation, and API validation milestones. A feature branch was created and integrated into develop before the stable main branch was updated.

4.1 Commit History

The evaluated commit sequence is:

Initial Smart Dairy API setup



Add project documentation



Validate API endpoints

The commit messages are short, action-oriented, and communicate the purpose of each milestone.



```
Git Commit History Review
$ git --no-pager log --online --graph --decorate --all
```

Figure 4.1 – Git Commit History and Branch Review

4.2 Branching Strategy

The project uses main as the stable branch, develop as the integration branch, and feature/health-monitoring for isolated feature work. This is suitable for the prototype because it separates stable code from ongoing development.

4.3 Collaboration Evaluation

Feature branches allow different contributors to work on health, feeding, milk, analytics, and AI modules independently.

Meaningful commits provide traceability and make code review easier.

A develop branch can act as an integration checkpoint before stable release.

A remote GitHub repository should be connected for actual team collaboration; no remote repository URL was supplied for this evaluation.

Branch protection and pull-request review should be introduced for a production project.

4.4 Version Control Improvement Recommendations

Use pull requests for feature integration.

Require review before merging into main.

Use issue references in commits where appropriate.

Add automated tests to the CI pipeline.

Use tags or releases for stable versions.

Never commit credentials, local environments, or generated secrets.

5. CODE TESTING AND VALIDATION

Testing was performed against the actual FastAPI application logic using the implemented endpoints. The validation evidence demonstrates successful responses for health monitoring, cattle records, disease-risk prediction, and farm analytics.

5.1 Test Cases

Test ID	Endpoint	Input / Purpose	Expected Result	Observed
TC-01	GET /health	Verify service availability	HTTP 200 and healthy status	Pass
TC-02	GET /cattle	Retrieve cattle sample records	HTTP 200 with cattle list	Pass
TC-03	POST /disease/predict	High temperature and heart rate for C002	High-risk response	Pass
TC-04	GET /analytics/summary	Retrieve farm-level indicators	HTTP 200 with summary	Pass

5.2 Test Results

The disease prediction test used cattle C002 with temperature 40.1°C and heart rate 105 bpm. The endpoint returned a High risk classification and recommended veterinary review. This verifies the implemented rule path, not the accuracy of a trained AI model.

```
Code Review - API Test Execution

$ application validation using FastAPI TestClient
GET /health
HTTP 200
{"status":"healthy","service":"Smart Dairy Farm Management System","version":"1.0.0"}

GET /cattle
HTTP 200
[{"id":"C001","name":"Lakshmi","status":"Normal","last_milk_litres":18.5},{id":"C002","name":"Nandini","status":"Monitor","last_milk_litres":15.

POST /disease/predict
HTTP 200
{"cattle_id":"C002","risk_level":"High","prediction":"Possible fever or infection - veterinary review recommended"}

GET /analytics/summary
HTTP 200
{"date":"2026-08-13","total_cattle":120,"healthy_cattle":112,"monitoring_required":8,"average_milk_litres_per_cow":17.8,"estimated_daily_milk_lit
```

Figure 5.1 – API Test Execution and Validation Results

5.3 Validation Limitations

The current test set is small and primarily validates endpoint behavior. No automated pytest suite is currently included in the prototype. No performance, load, security, or concurrency testing has been demonstrated. Disease prediction requires real labeled livestock data and ML evaluation before operational use. Database persistence and IoT sensor connectivity are not part of the current prototype.

6. REPOSITORY DOCUMENTATION

The repository contains a README with basic project purpose, installation, execution, and endpoint information. This provides a useful starting point, but the documentation should be expanded to meet the needs of a new developer or evaluator.

6.1 Documentation Strengths

Project purpose is clearly stated.

Basic run instructions are provided.

Main API endpoints are listed.

The repository is small enough for the setup process to be understood quickly.

6.2 Documentation Gaps

Documentation Area	Current State	Recommended Improvement
Installation	Basic dependency and Uvicorn commands	Add Python version, virtual environment, Docker, and troubleshooting steps.
Usage	Endpoint list	Add request/response examples and sample JSON payloads.
Architecture	Not fully documented in README	Add component and data-flow diagrams.
Testing	Limited in README	Add test commands and expected results.
Configuration	Minimal	Document ports, environment variables, and future secrets handling.
Deployment	Docker files exist	Explain docker build, docker compose, logs, and health verification.

6.3 Dependency Review

The requirements file contains FastAPI, Uvicorn, and Pydantic. The list is minimal and appropriate for the current API prototype. For reproducible production builds, dependency versions should be pinned or managed through a lock mechanism.

7. CODE REVIEW FINDINGS AND RECOMMENDATIONS

7.1 Strengths Identified

1. The API has a clear and understandable purpose.
2. FastAPI provides automatic API documentation and a modern REST implementation.
3. Pydantic models provide structured request validation.
4. Endpoint names are readable and map to clear business functions.
5. Docker configuration provides a reproducible runtime foundation.
6. Git branching and meaningful commits support maintainability.
7. The prototype has executable validation evidence.

7.2 Areas Requiring Improvement

1. Split the monolithic main.py into routers, services, schemas, and models.
2. Replace hard-coded sample data with persistent database storage.
3. Replace threshold-only disease logic with a validated trained ML model.
4. Add automated unit and integration tests.
5. Add authentication and role-based authorization.
6. Add structured logging, error handling, and input constraints.
7. Pin dependencies and add CI checks for formatting, linting, and tests.
8. Add real IoT integration and sensor-data preprocessing.
9. Add monitoring, alerts, and audit trails.
10. Expand README documentation and deployment instructions.

7.3 Maintainability Assessment

Area	Assessment	Priority
Readability	Good for prototype	Medium
Modularity	Needs refactoring	High
Validation	Basic Pydantic validation	High
Testing	Manual/prototype validation only	High
Documentation	Basic but clear	Medium
Version Control	Good local prototype workflow	Medium
Deployment	Docker-ready prototype	Medium
AI Readiness	Rule-based foundation only	High

8. REPOSITORY MAINTENANCE AND COLLABORATION

Repository maintenance should evolve alongside the application. As more developers contribute, consistent branching, pull-request review, issue tracking, automated testing, and documentation become increasingly important.

8.1 Recommended Team Workflow

Issue / Requirement



Feature Branch



Implementation



Unit Tests + Linting



Pull Request



Code Review



Develop



Release Validation



Main

8.2 Suggested Repository Policies

Protect main from direct unreviewed pushes.

Require successful automated tests before merging.

Use pull requests for all feature integration.

Maintain a consistent commit-message convention.

Keep README and API documentation synchronized with implementation.

Review dependencies regularly for compatibility and security.

Store environment-specific secrets outside source control.

8.3 Future Repository Modules

When the project becomes a full-stack smart dairy platform, the repository can be expanded with separate frontend, backend, AI/ML, database, IoT integration, tests, deployment, and infrastructure modules. This will improve separation of responsibilities and make parallel development easier.

9. QUALITY ATTRIBUTE ANALYSIS

Although the assessment focuses on code review and repository evaluation, the prototype can also be assessed against important quality attributes relevant to a smart dairy platform.

Quality Attribute	Current Observation	Future Target
Maintainability	Simple but monolithic source file	Modular services and automated quality checks
Reliability	Basic health endpoint and deterministic rules	Database backups, monitoring, retries, and tested ML pipeline
Scalability	Single FastAPI service	Multiple services and scalable deployment
Security	No authentication in prototype	Authentication, authorization, encrypted secrets
Performance	Suitable for prototype API calls	Load testing and optimized data processing
Usability	API-oriented	Farmer dashboard and mobile-friendly interface
Testability	Endpoint behavior can be validated	Comprehensive automated unit/integration tests

9.1 Architectural Justification

FastAPI is suitable for the prototype because it supports REST APIs, structured validation through Pydantic, and automatic OpenAPI documentation. Docker provides runtime consistency, while Git provides traceable source evolution. These technologies form a practical foundation for later integration with a database, IoT gateway, trained AI model, and farmer-facing interface.

10. OVERALL EVALUATION AND CONCLUSION

The Code Review and Repository Evaluation shows that the Intelligent AI-Based Smart Dairy Farm Management System is a clear and functional prototype with a suitable technology foundation. The application demonstrates understandable API design, structured request models, basic health and analytics services, and a prototype disease-risk workflow.

The strongest areas are readability, straightforward endpoint organization, Docker readiness, and a simple Git workflow. The main weaknesses are the concentration of logic in one file, hard-coded data, absence of automated tests, limited documentation, lack of authentication, and the use of threshold-based disease prediction instead of a validated machine-learning model.

The prototype should therefore be considered a foundation rather than a production-ready dairy management system. The recommended next stage is modular refactoring followed by database integration, automated testing, real IoT data ingestion, trained AI model development, security controls, and a farmer-oriented dashboard.

10.1 Final Recommendations

1. Refactor the application into domain-based modules.
2. Introduce automated testing and CI validation.
3. Add a database for persistent cattle, sensor, feeding, and milk records.
4. Develop and validate an actual machine-learning disease prediction model.
5. Integrate wearable and environmental IoT sensors.
6. Add authentication, authorization, logging, and monitoring.
7. Expand the README with complete installation, usage, testing, and deployment instructions.
8. Use pull requests and protected branches for collaborative development.
9. Add dashboards and alerts for practical farmer usage.
10. Conduct security, performance, and real-world farm validation before production deployment.

10.2 Assessment Evidence Summary

Repository structure screenshot included.

Git commit history screenshot included.

Source-code review screenshot included.

API test execution screenshot included.

Architecture/component diagram included.

Code snippets and review observations included.

Actual prototype endpoints were validated for expected response behavior.

Repository reference: The evaluation uses a local Git repository created for the prototype. A public GitHub URL was not supplied; therefore, no external repository link is claimed in this report.